

Build Governance Platform Development Review Findings

Version 1.0 | 17 September 2026 | Open findings for remediation

Prepared for Freston Kenny Adedeme and the development team. Review prepared by ChatGPT from the supplied code snapshot; this is not an independent human sign-off or a production approval.

Review conclusion

Development has progressed substantially beyond the initial repository skeleton. The backend contains identity, tenancy, templates, projects, evidence, decisions, integrity, export/import and draft support. However, approval controls and data portability still have material gaps. Correct and verify the high-priority findings before relying on the system for real governance decisions. Frontend design may continue alongside that work.

Finding	Priority	Status
BGP-F01 MFA enrolment substitutes for session authentication	High	Open
BGP-F02 Compensating review lacks independent confirmation	High	Open
BGP-F03 Decision checks lack coordinated concurrency protection	High	Open
BGP-F04 Export and import do not preserve the full record	High	Open
BGP-F05 Repository setup and status guidance are stale	Medium	Open

Evidence and limitations

The review inspected source code, migrations, test definitions, CI configuration and project notes. Syntax parsing passed for 52 Python files; 110 test functions were found. These are structural checks, not executed tests. TRACKER.md reports 112 passing test cases and a green GitHub Actions run, but those results were not independently reproduced. Function counts and executed-case counts can differ, for example through parametrisation.

Required runtime dependencies were unavailable in the review environment. The private repository returned HTTP 404 through the connected access, so live CI results, branch protections and pull-request reviews were not verified. Findings are based on source inspection, not demonstrated runtime exploits. This targeted review is not an exhaustive security audit.

BGP F01 MFA must be verified for the current session

High | REQ-002 and REQ-038 | WP03 and WP07 | Status: Open

Code evidence

backend/app/routers/auth.py:51 login; backend/app/security.py:39 create_session_token;
backend/app/deps.py:67 require_mfa; backend/app/routers/decisions.py _require_decision_authority;
backend/app/routers/mfa.py:43 enroll.

What the source shows

Login verifies the password and immediately issues a session token. The token carries a user ID, without a second-factor verification state. Approval guards check the account-level mfa_enabled flag. That flag shows enrolment, not that the current session completed a second factor. The enrolment endpoint also replaces the existing seed using a password-authenticated session, without checking the existing second factor.

Why it matters

For an already-enrolled approver, someone possessing the password could obtain a session that satisfies the current MFA guard. Password-only access could also replace the enrolled factor. Encryption of the stored seed does not resolve either authentication gap.

Required correction

Use a limited pre-authentication session after the password step. Grant approval-capable authentication only after a valid second factor, with session-bound verification time and an expiry policy. Require appropriate reauthentication or a verified recovery process before replacing an enrolled factor. Invalidate affected sessions after factor reset, recovery or relevant security changes.

Verification before closure

1. An enrolled approver who supplies only a correct password cannot record or supersede a decision.
2. A valid second factor enables only the verified session; a separate password-only session remains blocked.
3. Expired or revoked MFA assurance and a recovered/reset factor cannot retain approval access.
4. Replacing an enrolled factor without the required existing-factor verification or approved recovery path is rejected.

Closure evidence and responsibility

Supply authentication regression tests and endpoint-level results from a clean environment. Record the selected session design and security review. Assign an identity/security lead; no individual assignment is made by this report.

BGP F02 Record an actual compensating review

High | REQ-006 | WP07 | Status: Open

Code evidence

backend/app/routers/decisions.py:252 _check_separation_of_duties; SeparationOverrideIn; _record_decision.

What the source shows

The deciding user submits reviewer_user_id and a note. The check requires a different person with a project approval role, but it does not require a review action authenticated as that person or a stored approval bound to the evidence. The self-only check also uses current evidence owners, rather than establishing all relevant preparation activity from revision attribution.

Why it matters

Naming an eligible colleague can be treated as compensating review even if that colleague never reviewed the evidence. Changing ownership can also make the preparer test unreliable. The resulting record may imply independence that has not been demonstrated.

Required correction

Create a separate review record made by the authenticated reviewer. Bind it to the occurrence, evidence manifest, review scope and applicable authority. Validate active membership, independence and any required authentication when the decision is committed. Establish preparer identity from attributable work, not only the current owner field. Changes to reviewed evidence must require a fresh review.

Verification before closure

1. A reviewer ID and note, without that reviewer's recorded action, cannot enable self-only approval.
2. A review authenticated as an independent, authorised reviewer permits the intended scoped override.
3. A revoked reviewer, changed evidence manifest or review for another occurrence is rejected.
4. Reassigning an item cannot erase its preparation history or falsely satisfy the separation rule.

Closure evidence and responsibility

Provide the review-record design, migration, negative and positive tests, and an independent human review of the separation rule. Assign the backend and governance leads; appointments remain a project action.

BGP F03 Protect decision checks against concurrent changes

High | REQ-024 and REQ-023 | WP06 and WP07 | Status: Open

Code evidence

backend/app/routers/decisions.py:318 _record_decision and :477 create_decision;
backend/app/routers/projects.py:527 create_evidence_revision; backend/app/db.py.

What the source shows

Decision processing reads authority and readiness, compares a manifest and later writes the decision. The inspected paths do not use a shared locking protocol or explicitly configured serializable transactions covering the relevant evidence and authority changes. Evidence revision writes compare base_revision in application code. Transactional writes and uniqueness constraints alone do not establish atomic validation of all decision inputs.

Why it matters

An evidence update or authority change may occur after the decision checks but before commit. The decision could then acknowledge a state that is no longer current. This is a source-identified race risk; it was not reproduced with overlapping database sessions during this review.

Required correction

Select a concurrency design that makes evidence writers, decision writers and authority changes participate in compatible protection. Options include coordinated occurrence/authority locks or serializable transactions with bounded conflict handling. Re-read current authority and the bound evidence inside that protection. Handle duplicate idempotency keys and conflicting changes with deterministic outcomes, not unhandled database errors.

Verification before closure

1. Using two PostgreSQL sessions and explicit barriers, change evidence between readiness evaluation and commit; the stale decision must not succeed.
2. Revoke membership or approval authority during the same interval; the operation must serialize safely or reject the now-invalid decision.
3. Concurrent submissions with the same idempotency key produce one durable result; conflicting payloads are rejected.
4. Concurrent superseding decisions cannot create conflicting successor histories; conflicts have a defined response.

Closure evidence and responsibility

Provide the transaction design, database isolation settings and repeatable real-PostgreSQL concurrency tests, including assertions about final records. Single-threaded SQLite tests are not sufficient closure evidence.

BGP F04 Preserve history through export and import

High | REQ-030 and REQ-019 | WP09 | Status: Open

Code evidence

backend/app/routers/exports.py:87 `_build_archive` and :325 `commit_import`; Blueprint v0.2 Sections 5.6 and 6.2.

What the source shows

The archive builder exports current evidence items, but not their complete revision history and associated source-version/hash fields. Decision export omits the full manifest, conditions and compensating-review details. Import creates current evidence as revision 1 and keeps exported decisions, exceptions and audit material as historical archive content instead of reconstructing a complete usable historical record.

Why it matters

The exported record is already incomplete before import. A clean-instance transfer cannot faithfully preserve all evidence revisions and decision context. This does not satisfy the approved lossless portability requirement. Keeping unknown actors from becoming live authorised users is correct and must remain in place.

Required correction

Version the archive schema and include every required revision, manifest, condition, supersession link and attribution record. Preserve source identity through a stable mapping. Provide historical actor records that do not confer login or approval rights. Historical decisions may remain read-only, but they must be faithfully retrievable, linked and preserved through subsequent exports. Validate references and reconcile record counts and content.

Verification before closure

1. Export a fixture with multiple evidence revisions, source hashes, a conditional decision, an override, an exception and a superseding decision.
2. Import into a clean instance and verify history, relationships, attribution and decision context without activating imported authority.
3. Export the imported data again and reconcile semantic content and stable source mappings; regenerated local IDs alone must not hide missing data.
4. Reject broken references or corrupted archive content without leaving a partially usable project.

Closure evidence and responsibility

Provide a field-level round-trip coverage matrix, versioned fixtures, clean-instance import evidence and re-export reconciliation results. Assign the data/backend lead. Reduced current-state-only transfer must not be labelled compliant without an approved requirement change.

BGP F05 Bring the setup guide up to date

Medium | REQ-049 and WP01 delivery controls | Status: Open

Code evidence

README.md:7 status, repository layout and build instructions, and :59 endpoint claim; backend/app/main.py; backend/app/core/config.py; backend/alembic/versions/.

What the source shows

The README says that identity, tenancy, template, evidence and decision code do not exist, describes a health-check-only skeleton and says WP01 needs no database. The repository now includes those backend modules and nine migrations. Current setup guidance does not provide a complete reproducible path for the evolved application.

Why it matters

A new developer may misjudge progress, skip database preparation or use unsuitable configuration defaults. In particular, the current MFA encryption key has a random per-process default; developers using a persistent database need clear key-persistence instructions to avoid losing access to enrolled secrets after restart.

Required correction

Update status, supported endpoints and prerequisites. Document database roles, connection variables, migration commands, synthetic fixture setup, persistent development keys, test commands and known limitations. Include a safe configuration example without real secrets. Distinguish prototype support from verified completion and keep the no-frontend limitation explicit.

Verification before closure

1. A developer follows the README from a clean checkout, prepares the database, applies all migrations and starts the service.
2. The documented test commands collect and run the intended unit and PostgreSQL suites, with skipped tests clearly explained.
3. A restart with the documented persistent configuration preserves expected session/key behaviour and decrypts existing development MFA seeds.
4. README status and endpoint descriptions agree with the actual application and tracker.

Closure evidence and responsibility

Provide a clean-setup transcript identifying the source commit, runtime versions and exact commands. Assign the delivery lead. Updating prose alone is insufficient without checking the setup procedure.

Follow through and source record

Recommended correction sequence

Address BGP-F01 and BGP-F02 first, then BGP-F03 before treating approval outcomes as reliable. Complete BGP-F04 before claiming full data portability. Update BGP-F05 alongside the code changes so the corrected system can be reproduced. These priorities are review recommendations, not a new project gate decision.

Closing a finding

For each finding, record the assigned person, remediation commit or pull request, test IDs, execution environment, result, evidence location, reviewer and closure date. Mark it ready for verification after implementation; close it only after the relevant evidence has been checked. No finding is closed by this report, and no existing tracker result has been changed.

Other outstanding development work

The snapshot has no frontend. Accessibility and end-to-end user journeys therefore remain unverified. Decision durability and independent integrity custody remain open design/implementation work, as acknowledged in `app/main.py` and `TRACKER.md`. The tracker names Freston Kenny Adedeme as delivery lead and Milton as reviewer, but explicitly says reviewer acceptance, independence and actual code review have not been confirmed. A named reviewer is not a completed review.

Source identity

Archive: `pm-sdlc-tracker-bgp-master.zip`. Its ZIP comment identifies commit `a08ae1f17bdd358879365ff2d31fc5569bc8171f`. This identifier was read from the archive; it was not independently matched to the inaccessible remote.

Archive SHA-256: `1bda6f89ab05766ff240a024f830b6aa039aea6cdefe2661a98464adb76abd3a`

Repository recorded in `TRACKER.md`: <https://github.com/SageCarter98/pm-sdlc-tracker-bgp>. Reported CI evidence: <https://github.com/SageCarter98/pm-sdlc-tracker-bgp/actions/runs/35185824036>. These are source-reported references, not independently verified successful runs.

Baseline and scope control

Requirement references use the repository's approved Blueprint v0.2 and `Requirements_Catalogue.json`. Project Directive v1.2 remains the controlling project directive. KenAddme standing PM and SDLC frameworks remain institutional standards, not project-specific approvals. This report documents five findings from the 17 September review; it does not revise those baselines, certify compliance, authorise production or modify source code.

Line references identify the reviewed snapshot and may move in later revisions. Function names provide a second locator. High priority means a material approval-control or historical-record risk; Medium means a material reproducibility/status risk. No CVSS score or observed exploitation is asserted.